OneRec: Unifying Retrieve and Rank with Generative Recommender and Preference Alignment

Jiaxin Deng*
KuaiShou Inc.
Beijing, China
dengjiaxin03@kuaishou.com

Shiyao Wang*
KuaiShou Inc.
Beijing, China
wangshiyao08@kuaishou.com

Kuo Cai*
KuaiShou Inc.
Beijing, China
caikuo@kuaishou.com

Lejian Ren*
KuaiShou Inc.
Beijing, China
renlejian@kuaishou.com

Qigen Hu*
KuaiShou Inc.
Beijing, China
huqigen03@kuaishou.com

Weifeng Ding*
KuaiShou Inc.
Beijing, China
dingweifeng@kuaishou.com

Qiang Luo*
KuaiShou Inc.
Beijing, China
luoqiang@kuaishou.com

Guorui Zhou*†
KuaiShou Inc.
Beijing, China
zhouguorui@kuaishou.com

Abstract

Recently, generative retrieval-based recommendation systems (GRs) have emerged as a promising paradigm by directly generating candidate videos in an autoregressive manner. However, most modern recommender systems adopt a retrieve-and-rank strategy, where the generative model functions only as a selector during the retrieval stage. In this paper, we propose **OneRec**, which replaces the cascaded learning framework with a unified generative model. *To the best of our knowledge, this is the first end-to-end generative model that significantly surpasses current complex and well-designed recommender systems in real-world scenarios.* Specifically, OneRec includes: 1) **an encoder-decoder structure**, which encodes the user's historical behavior sequences and gradually decodes the videos that the user may be interested in. We adopt sparse Mixture-of-Experts (MoE) to scale model capacity without proportionally increasing computational FLOPs. 2) **a session-wise generation approach**. In contrast to traditional next-item prediction, we propose a session-wise generation, which is more elegant and contextually coherent than point-by-point generation that relies on hand-crafted rules to properly combine the generated results. 3) **an Iterative Preference Alignment module** combined with Direct Preference Optimization (DPO) to enhance the quality of the generated results. Unlike DPO in NLP, a recommendation system typically has only one opportunity to display results for each user's browsing request, making it impossible to obtain positive and negative samples simultaneously. To address this limitation, We design a reward model to simulate user generation and customize the sampling strategy according to the attributes of the recommendation system's online learning. Extensive experiments have demonstrated that a limited number of DPO samples can align user interest preferences and significantly improve the quality of generated results. We deployed OneRec in the main scene of Kuaishou, a short video recommendation platform with hundreds of millions of daily active users, achieving a 1.6% increase in watch-time, which is a substantial improvement.

CCS Concepts

• **Information systems** → **Computational advertising**; **Multimedia information systems**.

Keywords

Generative Recommendation, Autoregressive Generation, Semantic Tokenization, Direct Preference Optimization

ACM Reference Format:

Jiaxin Deng, Shiyao Wang, Kuo Cai, Lejian Ren, Qigen Hu, Weifeng Ding, Qiang Luo, and Guorui Zhou. 2018. OneRec: Unifying Retrieve and Rank with Generative Recommender and Preference Alignment. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 10 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

To balance efficiency and effectiveness, most modern recommender systems adopt a cascade ranking strategy[6, 26, 34, 43]. As illustrated in Figure 1(b), a typical cascade ranking system employs a three-stage pipeline: recall [6, 19, 54], pre-ranking [28, 46], and ranking [2, 3, 15, 16, 33, 52, 53]. Each stage is responsible for selecting the top-k items from the received items and passing the results to the next stage, collectively balancing the trade-off between system response time and sorting accuracy.

*Equal contribution.
†Corresponding author.

Conference acronym 'XX, Woodstock, NY

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

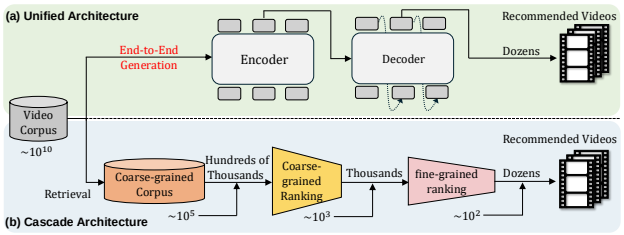

Figure 1: (a) Our proposed unified architecture for end-to-end generation. (b) A typical cascade ranking system, which includes three stages from the bottom to the top: Retrieval, Pre-ranking, and Ranking.

Although efficient in practice, existing methods typically treat each ranker independently, where the effectiveness of each isolated stage serves as the upper bound for the subsequent ranking stage, thereby limiting the performance of the overall ranking system. Despite various efforts [11, 13, 18, 20, 34, 44] to improve overall recommendation performance by enabling interaction among rankers, they still maintain the traditional cascade ranking paradigm. Recently, generative retrieval-based recommendation systems (GRs) [36, 45, 51] have emerged as a promising paradigm by directly generating the identifier of a candidate item in an autoregressive sequence generation manner. By indexing items with quantized semantic IDs that encode item semantics [24], recommenders can leverage the abundant semantic information within the items. The generative nature of GRs makes them suitable for directly selecting candidate items through beam search decoding and producing more diverse recommendation results. However, current generative models only act as selectors in the retrieval stage, as their recommendation accuracy does not yet match that of well-designed multiple cascade rankers.

To address the above challenges, we propose a unified end-to-end generative framework for single-stage recommendation named **OneRec**. *First*, we present an encoder-decoder architecture. Taking inspiration from the scaling laws observed in training large language models, we find that scaling the capacity of recommendation models also consistently improves the performance. So we scale up the model parameters based on the structure of MoE [7, 9, 55], which significantly improves the model's ability to characterize user interests. *Second*, unlike the traditional point-by-point prediction of the next item, we propose a session-wise list generation approach that considers the relative content and order of the items within each session. The point-by-point generation method necessitates hand-craft strategies to ensure coherence and diversity in the generated results. In contrast, the session-wise learning process enables the model to autonomously learn the optimal session structure by feeding it preferred data. *Last but not least*, we explore preference learning by using direct preference optimization (DPO) [35] to further enhance the quality of the generated results. For constructing preference pairs, we take inspiration from hard negative sampling [37] by creating self-hard rejected samples from the beam search results rather than random sampling. We propose an Iterative Preference Alignment (IPA) strategy to rank the sampled responses based on scores provided by the pre-trained reward model (RM), identifying the best-chosen and worst-rejected samples. Our experiments on large-scale industry datasets show the superiority of the proposed method. We also conduct a series of ablation experiments to demonstrate the effectiveness of each module in detail. The main contributions of this work are summarized as follows:

- To overcome the limitations of cascade ranking, we introduce OneRec, a single-stage generative recommendation framework. To the best of our knowledge, this is one of the first industrial solutions capable of handling item recommendations with a unified generation model, significantly surpassing the traditional multi-stage ranking pipeline.
- We highlight the necessity of model capacity and contextual information of target items through a session-wise generation manner, which enables more accurate predictions and enhances the diversity of generated items.
- We propose a novel self-hard negative samples selection strategy based on personalized reward model. With direct preference optimization, we enhance OneRec's generalization across a broader range of user preference. Extensive offline experiments and online A/B testing demonstrates their effectiveness and efficiency.

2 Related Work

2.1 Generative Recommendation

In recent years, with the remarkable progress in generative models, generative recommendation has received increasing attention. Unlike traditional embedding-based retrieval methods which largely rely on a two-tower model for calculating the ranking score for each candidate item and utilize an effecient MIPS or ANN [14, 17, 21, 31, 38] search system for retrieving top-k relevant items. Generative Retrieval (GR) [41] method formulates the problem of retrieving relevant documents from the database as a sequence generation task which generate the relevant document tokens sequentially. The document tokens can be the document titles, document IDs or pre-trained semantic IDs [42]. GENRE [8] first adopts the transformer architecture for entity retrieval, generating entity names in an autoregressive fashion based on the conditioned context. DSI [42] first proposes the concept of assigning structured semantic IDs to documents and training encoder-decoder models for generative document retrieval. Following this paradigm, TIGER [36] introduces the formulation of generative item retrieval models for recommender systems.

In addition to the generation framework, how to index items has also attracted increasing attention. Recent research focuses on the semantic indexing technique [12, 36, 42], which aims to index items based on content information. Specifically, TIGER [36] and LC-Rec [51] apply residual quantization (RQ-VAE) to textual embeddings derived from item titles and descriptions for tokenization. Recforest [12] utilizes hierarchical k-means clustering on item textual embeddings to obtain cluster indexes as tokens. Furthermore, recent studies such as EAGER [45] explore integrating both semantic and collaborative information into the tokenization process.

2.2 Preference Alignment of Language Models

During the post-training [10] phase of Large Language Models (LLMs), Reinforcement Learning from Human Feedback (RLHF) [32, 39] is a prevalent method in aligning LLMs with human values by employing reinforcement learning techniques guided by reward models that represent human feedback. However, RLHF suffers from

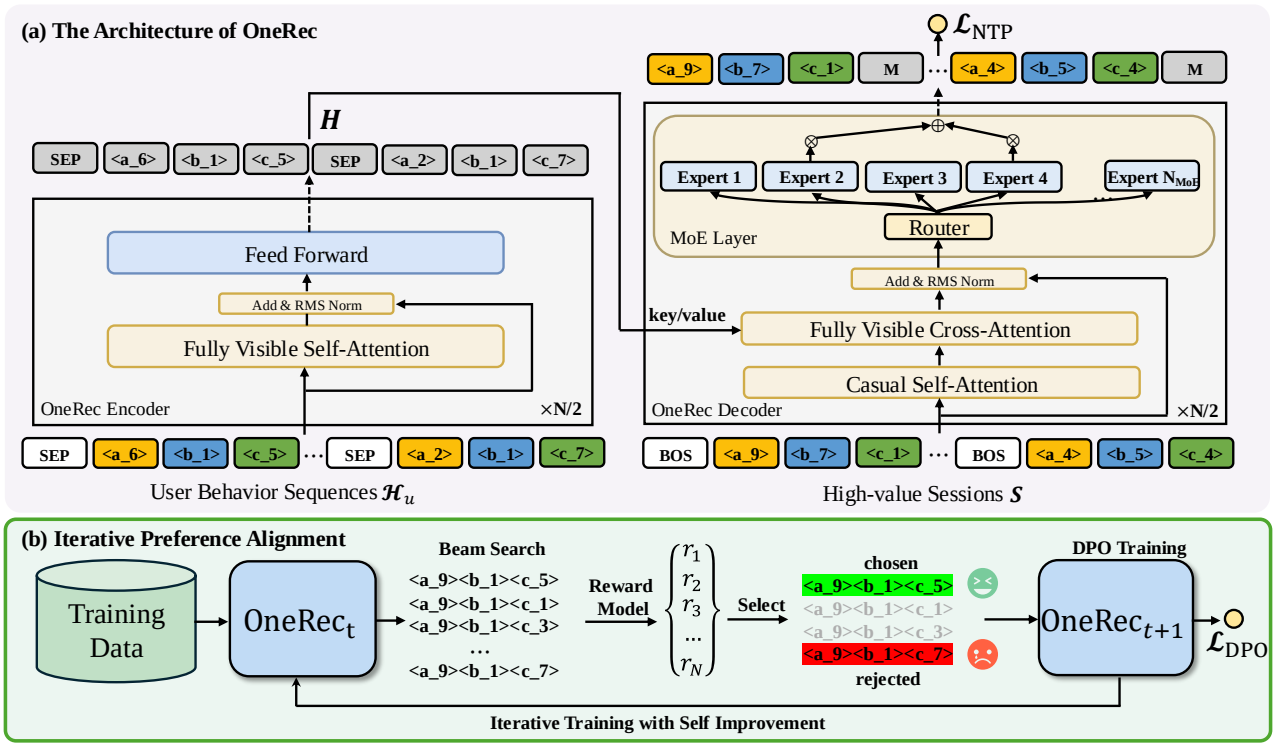

Figure 2: The overall framework of OneRec, consists of two stages: (i) the session training stage which train OneRec with session-wise data; (ii) the IPA stage which utilizes iterative direct preference optimization with self-hard negatives.

instability and inefficiency. Direct Preference Optimization (DPO) [35] is proposed which derives the optimal policy in closed form and enables direct optimization using preference data. Apart from that, several variants have been proposed to further improve the original DPO. For example, IPO [1] bypasses two approximations in DPO with a general objective. cDPO [35] alleviates the influence of noisy labels by introducing a hyperparameter ϵ. rDPO [5] designs an unbiased estimate of the original Binary Cross Entropy loss. Other variants including CPO [47], simDPO [5], also enhance or expand DPO in various aspects. However, unlike traditional NLP scenarios where preference data is explicitly annotated through humans, preference learning in recommendation systems faces a unique challenge because of the sparsity of user-item interaction data. This challenge results in adapting DPO for recommendation are still largely unexplored. Different from S-DPO which focuses on incorporating multiple negatives in user preference data for LM-based recommenders, we train a reward model and based on the scores from reward model we choose personalized preference data for different users.

3 Methods

In this section, we propose OneRec, an end-to-end framework that generates target items through a single-stage retrieval manner. In Section 3.1, we first introduce the feature engineering for the single-stage generative recommendation pipeline in industrial applications. Then, in Section 3.2, we formally define the session-wise generative tasks and present the architecture of our proposed OneRec model. Finally, we elaborate on the model's capability with a personalized reward model for self-hard negative sampling in Section 3.3, and demonstrate how we iteratively improve model performance through direct preference optimization. The overall framework of OneRec is illustrated in Figure 2.

3.1 Preliminary

In this section, we introduce the construction of the single-stage generative recommendation pipeline from the perspectives of feature engineering. For user-side feature, OneRec takes the positive historical behavior sequences $\mathcal{H}_u = \{v_1^h, v_2^h, \ldots, v_n^h\}$ as input, where v represent the videos that the user has effectively watched or interacted with (likes, follows, shares), and n is the length of behaviour sequence. The output of OneRec is a list of videos, consisting of a session $\mathcal{S} = \{v_1, v_2, ..., v_m\}$, where m is the number of videos within a session (the detailed definition of "session" can be found in Section 3.2).

For each video v_i, we describe them with multi-modal embeddings $\boldsymbol{e}_i \in \mathbb{R}^d$ which are aligned with the real user-item behaviour distribution [27]. Based on the pretrain multi-modal representation, existing generative recommendation frameworks [25, 36] use RQ-VAE [49] to encode the embedding into semantic tokens. However, such method is suboptimal due to the unbalanced code distribution which is known as the *hourglass phenomenon* [23]. We apply a multi-level balanced quantitative mechanism to transform the $\boldsymbol{e}_i$ with residual K-Means quantization algorithm[27]. At the first level ($l = 1$), the initial residual is defined as $\boldsymbol{r}_i^1 = \boldsymbol{e}_i$. At each level

Algorithm 1: Balanced K-means Clustering

Input: Item set $\mathcal{V}$, number of clusters K

Compute $w \leftarrow |\mathcal{V}|/K$

Initialize centroids $C_l = \{\boldsymbol{c}_1^l, \ldots, \boldsymbol{c}_K^l\}$ with random selection;

repeat

  Initialize unassigned set $\mathcal{U} \leftarrow \mathcal{V}$

  for *each cluster* $k \in \{1, \ldots, K\}$ **do**

    Sort $\mathcal{U}$ by ascending distance from centroid $\boldsymbol{c}_k^l$;

    Assign $\mathcal{V}_k \leftarrow \mathcal{U}[0 : w-1]$;

    Update centroid $\boldsymbol{c}_k^l \leftarrow \frac{1}{w} \sum_{\boldsymbol{r}^l \in \mathcal{V}_k} \boldsymbol{r}^l$;

    Remove assigned items $\mathcal{U} \leftarrow \mathcal{U} \setminus \mathcal{V}_k$;

  end

until *Assignment convergence*;

Output: Optimized codebook $C_l = \{\boldsymbol{c}_1^l, \ldots, \boldsymbol{c}_K^l\}$

l, we have a codebook $C_l = \{\boldsymbol{c}_1^l, ..., \boldsymbol{c}_K^l\}$, where K is the codebook size. The index of the closest centroid node embedding is generate through $\boldsymbol{s}_i^l = \arg\min_k \|\boldsymbol{r}_i^l - \boldsymbol{c}_k^l\|_2^2$ and for next level $l+1$ the residual is defined as $\boldsymbol{r}_i^{l+1} = \boldsymbol{r}_i^l - \boldsymbol{c}_{\boldsymbol{s}_i^l}^l$. So the corresponding codebook tokens are generated through hierarchical indexing:

$$
\begin{aligned}
\boldsymbol{s}_i^1 &= \arg\min_k \|\boldsymbol{r}_i^1 - \boldsymbol{c}_k^1\|_2^2, \quad \boldsymbol{r}_i^2 = \boldsymbol{r}_i^1 - \boldsymbol{c}_{\boldsymbol{s}_i^1}^1 \\
\boldsymbol{s}_i^2 &= \arg\min_k \|\boldsymbol{r}_i^2 - \boldsymbol{c}_k^2\|_2^2, \quad \boldsymbol{r}_i^3 = \boldsymbol{r}_i^2 - \boldsymbol{c}_{\boldsymbol{s}_i^2}^2 \\
&\vdots \\
\boldsymbol{s}_i^L &= \arg\min_k \|\boldsymbol{r}_i^L - \boldsymbol{c}_k^L\|_2^2
\end{aligned}
$$

where L is the total layers of sematic ID.

To construct a balanced codebook $C_l = \{\boldsymbol{c}_1^l, \ldots, \boldsymbol{c}_K^l\}$, we apply the Balanced K-means as detailed in **Algorithm 1** for itemset partitioning. Given the total video set $\mathcal{V}$, this algorithm partitions the set into K clusters, where each cluster contains exactly $w = |\mathcal{V}|/K$ videos. During iterative computation, each centroid is sequentially assigned its w nearest unallocated videos based on Euclidean distance, followed by centroid recalibration using mean vectors of assigned videos. The termination criterion is satisfied when cluster assignments reach convergence.

3.2 Session-wise List Generation

Different from traditional point-wise recommendation methods that only predict the next video, session-wise generation aims to generate a list of high-value sessions based on users' historical interaction sequences, which enables the recommendation model to capture the dependencies between videos in the recommended list. Specifically, a session refers to a batch of short videos returned in response to a user's request, typically consisting of 5 to 10 videos. The videos within a session generally take into account factors such as user interest, coherence, and diversity. We have devised several criteria to identify high-quality sessions, including:

- The number of short videos actually watched by the user within a session is greater than or equal to 5;
- The total duration for which the user watches the session exceeds a certain threshold;
- The user exhibits interaction behaviors, such as liking, collecting, or sharing the videos;

This approach ensures that our session-wise model learns from real user engagement patterns and captures more accurate contextual information within the session list. So the objective of our session-wise model $\mathcal{M}$ can be formalized as:

$$
\mathcal{S} := \mathcal{M}(\mathcal{H}_u) \tag{1}
$$

where $\mathcal{H}_u$ is represented from the remantic IDs: $\mathcal{H}_u = \{(\boldsymbol{s}_1^1, \boldsymbol{s}_1^2, \cdots, \boldsymbol{s}_1^L), (\boldsymbol{s}_2^1, \boldsymbol{s}_2^2, \cdots, \boldsymbol{s}_2^L), \cdots, (\boldsymbol{s}_n^1, \boldsymbol{s}_n^2, \cdots, \boldsymbol{s}_n^L)\}$ and $\mathcal{S} = \{(\boldsymbol{s}_1^1, \boldsymbol{s}_1^2, \cdots, \boldsymbol{s}_1^L), (\boldsymbol{s}_2^1, \boldsymbol{s}_2^2, \cdots, \boldsymbol{s}_2^L), \cdots, (\boldsymbol{s}_m^1, \boldsymbol{s}_m^2, \cdots, \boldsymbol{s}_m^L)\}$.

As shown in Figure 2 (a), consistent with the T5 [48] architecture, our model employs a transformer-based framework consisting of two main components: an encoder for modeling user historical interactions and a decoder for session list generation. Specifically, the encoder leverages the stacked multi-head self-attention and feed-forward layers to process the input sequence $\mathcal{H}_u$. We denote the encoded historical interaction features as $\boldsymbol{H} = Encoder(\mathcal{H}_u)$.

The decoder takes the semantic IDs of the target session as input and generates the target in an auto-regressive manner. To train a larger model at reasonable economic costs, for the feed-forward neural networks (FNNs) in the decoder, we adopt the MoE architecture [7, 9, 55] commonly used in Transformer-based language models and substitute the l-th FFN with:

$$
\begin{aligned}
\mathbf{H}_t^{l+1} &= \sum_{i=1}^{N_{\text{MoE}}} \left(g_{i,t}\, \text{FFN}_i\left(\mathbf{H}_t^l\right)\right) + \mathbf{H}_t^l, \\
g_{i,t} &= \begin{cases} s_{i,t}, & s_{i,t} \in \text{Topk}(\{s_{j,t} | 1 \le j \le N\}, K_{\text{MoE}}), \\ 0, & \text{otherwise}, \end{cases} \\
s_{i,t} &= \text{Softmax}_i\left(\mathbf{H}_t^{l\,T} \mathbf{e}_i^l\right),
\end{aligned} \tag{2}
$$

where N_{MoE} represents the total number of experts, $\text{FFN}_i(\cdot)$ is the i-th expert FFN, and $g_{i,t}$ denotes the gate value for the i-th expert. The gate value $g_{i,t}$ is sparse, meaning that only K_{MoE} out of N_{MoE} gate values are non-zero. This sparsity property ensures computational efficiency within an MoE layer and each token will be assigned to and computed in only K_{MoE} experts.

For training, we add a start token $\boldsymbol{s}_{[\text{BOS}]}$ at the beginning of codes to construct the decoder inputs with:

$$
\begin{aligned}
\bar{\mathcal{S}} = \{&\boldsymbol{s}_{[\text{BOS}]}, \boldsymbol{s}_1^1, \boldsymbol{s}_1^2, \cdots, \boldsymbol{s}_1^L, \boldsymbol{s}_{[\text{BOS}]}, \boldsymbol{s}_2^1, \boldsymbol{s}_2^2, \cdots, \boldsymbol{s}_2^L, \\
&\cdots, \boldsymbol{s}_{[\text{BOS}]}, \boldsymbol{s}_m^1, \boldsymbol{s}_m^2, \cdots, \boldsymbol{s}_m^L\}
\end{aligned} \tag{3}
$$

We utilize cross-entropy loss for next-token prediction on the sematic IDs of the target session. The NTP loss $\mathcal{L}_{\text{NTP}}$ is formulated as:

$$
\begin{aligned}
\mathcal{L}_{\text{NTP}} = -\sum_{i=1}^{m} \sum_{j=1}^{L} \log P(\boldsymbol{s}_i^{j+1} \mid [&\boldsymbol{s}_{[\text{BOS}]}, \boldsymbol{s}_1^1, \boldsymbol{s}_1^2, \cdots, \boldsymbol{s}_1^L, \cdots, \\
&\boldsymbol{s}_{[\text{BOS}]}, \boldsymbol{s}_i^1, \cdots, \boldsymbol{s}_i^j]; \Theta).
\end{aligned} \tag{4}
$$

After a certain amount of training on session-wise list generation task, we obtain the seed model $\mathcal{M}_t$.

Algorithm 2: Iterative Preference Alignment (IPA)

Input: Number of responses N, pretrained RM $R(\boldsymbol{u}, \mathcal{S})$, seed model $\mathcal{M}_t$, DPO ratio r_{DPO}, total epochs T and samples per epoch N_{sample}

for *epoch* $\leftarrow t$ **to** T **do**
  for *sample* $\leftarrow 1$ **to** N_{sample} **do**
    if *rand*() $< r_{\text{DPO}}$ **then**
      Generate N responses via $\mathcal{M}_t$:
      for $i \leftarrow 1$ **to** N **do**
        $\mathcal{S}_u^i \sim \mathcal{M}_t(\mathcal{H}_u)$;
        $r_u^i \leftarrow R(\boldsymbol{u}, \mathcal{S}_u^i)$;
      end
      Select the best and worst responses:
      $\mathcal{S}_u^w \leftarrow \mathcal{S}_u^{\arg\max_i r_u^i}$;
      $\mathcal{S}_u^l \leftarrow \mathcal{S}_u^{\arg\min_i r_u^i}$;
      Compute NTP and DPO loss:
      $\mathcal{L} \leftarrow \mathcal{L}_{\text{NTP}} + \lambda \mathcal{L}_{\text{DPO}}$;
    else
      Compute NTP loss:
      $\mathcal{L} \leftarrow \mathcal{L}_{\text{NTP}}$;
    end
    Update parameters:
    $\Theta \leftarrow \Theta - \alpha \nabla_\Theta \mathcal{L}$;
  end
  Update model snapshot: $\mathcal{M}_{t+1} \leftarrow \mathcal{M}_t$;
end

Output: Optimized parameters Θ

3.3 Iterative Preference Alignment with RM

The high-quality sessions defined in Section 3.2 provide valuable training data, enabling the model to learn what constitutes a good session, thereby ensuring the quality of generated videos. Building on this foundation, we aim to further enhance the model's ability by Direct Preference Optimization (DPO). In traditional natural language processing (NLP) scenarios, preference data is explicitly annotated by humans. However, preference learning in recommendation systems confronts a unique challenge due to the sparsity of user-item interaction data, which necessitates a reward model (RM). So we introduce a session-wise reward model in Section 3.3.1. Moreover, we improve the conventional DPO by proposing an iterative direct preference optimization that enables the model to self-improvement described in Section 3.3.2.

3.3.1 Reward Model Training. We use $R(\boldsymbol{u}, \mathcal{S})$ to denote the reward model which selects preference data for different users. Here, the output r represents the reward corresponding to user u's (usually represented by user behavior) preference on the session $\mathcal{S} = \{\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_m\}$. In order to equip the RM with the capacity to rank session, we first extract the target-aware representation $\boldsymbol{e}_i = \boldsymbol{v}_i \odot \boldsymbol{u}$ of each item $\boldsymbol{v}_i$ in $\mathcal{S}$, where $\odot$ represents the target-aware operation (such as target attention toward user behavior). So we get the target-aware representation $\boldsymbol{h} = \{\boldsymbol{e}_1, \boldsymbol{e}_2, \cdots, \boldsymbol{e}_m\}$ for session $\mathcal{S}$. Then the items within a session interact with each other through self-attention layers to fuse the necessary information among different items:

$$\boldsymbol{h}_f = \text{SelfAttention}(\boldsymbol{h}\boldsymbol{W}_s^Q, \boldsymbol{h}\,\boldsymbol{W}_s^K, \boldsymbol{h}\boldsymbol{W}_s^V) \tag{5}$$

Next we utilize different tower to make predictions on multi-target reward and the RM is pre-trained with abundant recommendation data:

$$\begin{aligned} &\hat{r}^{swt} = \texttt{Tower}^{swt}(\texttt{Sum}(\boldsymbol{h}_f)), \hat{r}^{vtr} = \texttt{Tower}^{vtr}(\texttt{Sum}(\boldsymbol{h}_f)), \\ &\hat{r}^{wtr} = \texttt{Tower}^{wtr}(\texttt{Sum}(\boldsymbol{h}_f)), \hat{r}^{ltr} = \texttt{Tower}^{ltr}(\texttt{Sum}(\boldsymbol{h}_f)), \\ &\quad \texttt{where} \quad \texttt{Tower}(\cdot) = \texttt{Sigmoid}(\texttt{MLP}(\cdot)) \end{aligned} \tag{6}$$

After getting all the estimated rewards $\hat{r}^{swt}, \ldots$ and the ground-truth labels $y^{swt}, \ldots$ for each session, we directly minimize the binary cross-entropy loss to train the RM. The loss function $\mathcal{L}_{\text{RM}}$ is defined as follows:

$$\mathcal{L}_{\text{RM}} = -\sum_{swt,\ldots}^{xtr} \left(y^{xtr} \log(\hat{r}^{xtr}) + (1 - y^{xtr}) \log(1 - \hat{r}^{xtr}) \right) \tag{7}$$

3.3.2 Iterative Preference Alignment. Based on pre-trained RM $R(\boldsymbol{u}, \mathcal{S})$ and current OneRec $\mathcal{M}_t$, we generate N different responses for each user by beam search:

$$\mathcal{S}_u^n \sim M_t(\mathcal{H}_u) \quad \text{for all } u \in \mathcal{U} \text{ and } n \in [N], \tag{8}$$

where we use $[N]$ to denote $\{1, 2, \ldots, N\}$.

Then we computes the reward r_u^n for each of these responses based on the RM $R(\boldsymbol{u}, \mathcal{S})$:

$$r_u^n = R(\boldsymbol{u}, \mathcal{S}_u^n) \tag{9}$$

Next we build the preference pairs $D_t^{\text{pairs}} = (\mathcal{S}_u^w, \mathcal{S}_u^l, \mathcal{H}_u)$ by choosing the winner response $(\mathcal{S}_u^w, \mathcal{H}_u)$ with the highest reward value and a loser response $(\mathcal{S}_u^l, \mathcal{H}_u)$ with the lowest reward value. Given the preference pairs, we can now train a new model M_{t+1} which is initialized from model M_t, and updated with a loss function that combines the DPO loss [35] for learning from the preference pairs. The loss corresponding to each preference pair is as follows:

$$\begin{aligned} \mathcal{L}_{\text{DPO}} &= \mathcal{L}_{\text{DPO}}(\mathcal{S}_u^w, \mathcal{S}_u^l | \mathcal{H}_u) \\ &= -\log \sigma \left(\beta \log \frac{M_{t+1}(\mathcal{S}_u^w | \mathcal{H}_u)}{M_t(\mathcal{S}_u^w | \mathcal{H}_u)} - \beta \log \frac{M_{t+1}(\mathcal{S}_u^l | \mathcal{H}_u)}{M_t(\mathcal{S}_u^l | \mathcal{H}_u)} \right). \end{aligned} \tag{10}$$

As shown in Algorithm 2 and Figure 2 (b), the overall procedure involves training a sequence of models $M_t, \ldots, M_T$. To mitigate the computational burden during beam search inference, we randomly sample only $r_{\text{DPO}} = 1\%$ data for preference alignment. For each successive model M_{t+1}, it initializes from previous model M_t and utilizes the preference data D_t^{pairs} generated by the M_t for training.

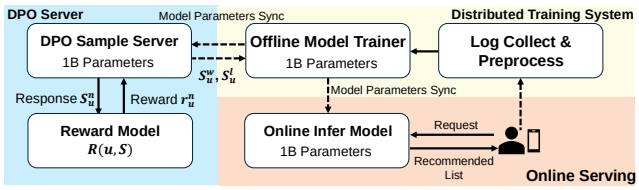

Figure 3: Framework of Online Deployment of OneRec.

Table 1: Offline performance of our proposed OneRec (green) with pointwise methods (brown), listwise methods (blue) and preference alignment methods (yellow). Best results are in bold, sub-optimal results are underlined. Metrics with ↑ indicate higher is better, while ↓ indicates lower is better.

Model	*Watching-Time Metrics*				*Interaction Metrics*			
	swt↑		vtr↑		wtr↑		ltr↑	
	mean	max	mean	max	mean	max	mean	max
Pointwise Discriminative Method								
SASRec	0.0375±0.002	0.0803±0.005	0.4313±0.013	0.5801±0.013	0.00294±0.001	0.00978±0.001	0.0314±0.002	0.0604±0.004
BERT4Rec	0.0336±0.002	0.0706±0.004	0.4192±0.014	0.5633±0.013	0.00281±0.001	0.00932±0.001	0.0316±0.002	0.0606±0.004
FDSA	0.0325±0.002	0.0683±0.005	0.4145±0.015	0.5588±0.014	0.00271±0.001	0.00921±0.001	0.0313±0.002	0.0604±0.003
Pointwise Generative Method								
TIGER-0.1B	0.0879±0.007	0.1286±0.010	0.5826±0.016	0.6625±0.017	0.00277±0.001	0.00671±0.001	0.0316±0.004	0.0541±0.007
TIGER-1B	0.0873±0.006	0.1368±0.010	0.5827±0.015	0.6776±0.015	0.00292±0.001	0.00758±0.001	0.0323±0.004	0.0579±0.008
Listwise Generative Method								
OneRec-0.1B	0.0973±0.010	0.1501±0.015	0.6001±0.021	0.6981±0.021	0.00326±0.001	0.00870±0.001	0.0349±0.009	0.0642±0.015
OneRec-1B	0.0991±0.008	0.1529±0.012	0.6039±0.020	0.7013±0.020	0.00349±0.001	0.00919±0.002	0.0360±0.005	0.0660±0.008
Listwise Generative Method with Preference Alignment								
OneRec-1B+DPO	0.1014±0.010	0.1595±0.015	0.6127±0.017	0.7116±0.016	0.00339±0.001	0.00896±0.001	0.0351±0.004	0.0644±0.008
OneRec-1B+IPO	0.0979±0.003	0.1528±0.005	0.6000±0.007	0.7012±0.007	0.00335±0.001	0.00905±0.001	0.0350±0.003	0.0654±0.004
OneRec-1B+cDPO	0.0993±0.006	0.1547±0.008	0.6030±0.011	0.7030±0.009	0.00339±0.001	0.00901±0.001	0.0355±0.006	0.0652±0.009
OneRec-1B+rDPO	0.1005±0.006	0.1555±0.008	0.6071±0.014	0.7059±0.011	0.00339±0.001	0.00899±0.001	0.0357±0.004	0.0657±0.006
OneRec-1B+CPO	0.0993±0.008	0.1538±0.012	0.6045±0.021	0.7029±0.018	0.00343±0.001	0.00911±0.002	0.0357±0.008	0.0659±0.014
OneRec-1B+simPO	0.0995±0.008	0.1536±0.013	0.6047±0.016	0.7022±0.015	0.00349±0.001	0.00918±0.001	0.0360±0.005	0.0659±0.008
OneRec-1B+S-DPO	0.1021±0.008	0.1575±0.013	0.6096±0.016	0.7070±0.015	0.00345±0.001	0.00909±0.001	0.0361±0.004	0.0659±0.008
OneRec-1B+IPA	**0.1025±0.009**	**0.1933±0.017**	**0.6141±0.020**	**0.7646±0.021**	**0.00354±0.001**	**0.00992±0.001**	**0.0397±0.004**	**0.1203±0.010**

4 System Deployment

OneRec has been successfully implemented in real-world industrial scenarios. Balancing stability and performance, we deploy the OneRec-1B for online services. As illustrated in Figure 3, our deployment architecture consists of three core components: 1) the training system, 2) the online serving system, and 3) the DPO sample server. The system processes collected interaction logs as training data, initially adopting the next token prediction objective $\mathcal{L}_{\text{NTP}}$ to train the seed model. After convergence, we add the DPO loss $\mathcal{L}_{\text{DPO}}$ for preference alignment, leveraging XLA and bfloat16 mixed-precision training to optimize computational efficiency and memory utilization. The trained parameters are synchronized to the online inference module and the DPO sampling server for real-time serving and preference-based data selection. To enhance inference performance, we implement two key optimizations: the key-value cache decoding mechanism combined with float16 quantization to reduce GPU memory overhead, and the beam search configuration with beam size of 128 to balance generation quality and latency. Additionally, thanks to the MoE architecture, during inference only 13% of the parameters are activated.

5 Experiment

In this section, we first compare OneRec with the point-wise methods and several DPO variations in offline settings. Then, we conduct some ablation experiments on our proposed module to verify the effectiveness of OneRec. Finally, we deploy OneRec to the online and conduct A/B test to further validate its performance on Kuaishou.

5.1 Experimental Settings

5.1.1 Implementation Details. Our model is trained using the Adam optimizer with an initial learning rate of 2×10^{-4}. We utilize NVIDIA A800 GPUs for OneRec optimization. The DPO sample ratio r_{DPO} is set to 1% throughout training and we generate $N = 128$ different responses for each user by beam search; The semantic identifier clustering process employs $K = 8192$ clusters for each codebook layer and the number of codebook layers is set to $L = 3$; The Mixture-of-Experts architecture contains $N_{\text{MoE}} = 24$ expert with $K_{\text{MoE}} = 2$ experts activated per forward pass through top-k selection; For session modeling, we consider $m = 5$ target session items and adopt $n = 256$ historical behavior as context.

5.1.2 Baseline Methods. We adopt the following representative recommendation models, DPO and its variants to serve as additional baselines for comparison. The baseline methods include:

- **SASRec** [22] employs a unidirectional Transformer architecture to capture sequential dependencies in user-item interactions for next-item prediction.
- **BERT4Rec** [40] leverages bidirectional Transformers with masked language modeling to learn contextual item representations through sequence reconstruction.
- **FDSA** [50] implements dual self-attention pathways to jointly model item-level transitions and feature-level transformation patterns in heterogeneous recommendation scenarios.
- **TIGER** [36] leverages hierarchical semantic identifiers and generative retrieval techniques for sequential recommendation through auto-regressive sequence generation.

- **DPO** [35] formalizes preference optimization with a closed-form reward function derived from human feedback data via implicit reward modeling.
- **IPO** [1] proposes a theoretically grounded preference optimization framework which bypass the approximations inherent in standard DPO.
- **cDPO** [30] introduces a robustness-aware variant incorporating a label flipping rate parameter ϵ to account for noisy preference annotations.
- **rDPO** [5] develops an unbiased loss estimator using importance sampling to reduce variance in preference optimization.
- **CPO** [47] unifies contrastive learning with preference optimization through joint training of sequence likelihood rewards and supervised fine-tuning objectives.
- **simPO** [29] conducts preference optimization by employing sequence-level reward margins while eliminating reference model dependencies through normalized probability averaging.
- **S-DPO** [4] adapts DPO for recommendation systems through hard negative sampling and multi-item contrastive learning to enhance ranking accuracy.

5.1.3 Evaluation Metric. We evaluate the model's performance with several key metrics. Each metric serves a distinct purpose in assessing different aspects of the model's output and we conduct the evaluation on a randomly sampled set of test cases in each iteration. To estimate the probabilities of various interactions for each specific user-session pair, we employ the pre-trained reward model to assess the value of recommended sessions. We calculate the mean reward for different target metrics, including session watch time (**swt**), view probability (**vtr**), follow probability (**wtr**) and like probability (**ltr**). Among these targets, swt and vtr are watching-time metrics, while wtr and ltr are interaction metrics.

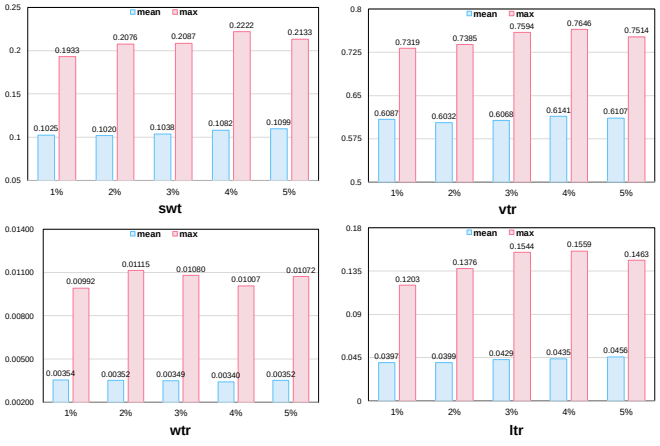

Figure 4: The ablation study on DPO sample ratio r_{DPO}. The results indicate that a 1% ratio of DPO training leads to significant gains but further increase the sample ratio results in limited improvements.

5.2 Offline Performance

Table 1 presents the comprehensive comparison between OneRec and various baselines. For watching-time metric we mainly care about the session watch time (**swt**) and like probability (**ltr**) in interaction metrics. Our result reveals three key observations:

First, the proposed session-wise generation approach significantly outperforms traditional dot-product-based methods and point-wise generation methods like TIGER. OneRec-1B achieves 1.78% higher maximum swt and 3.36% higher maximum ltr compared to TIGER-1B. This demonstrates the advantage of session-wise modeling in maintaining contextual coherence across recommendations, whereas point-wise methods struggle to balance coherence and diversity in generated outputs.

Second, a small ratio of DPO training yields substantial gains. With only 1% DPO training ratio (r_{DPO}), OneRec-1B+IPA surpasses the base OneRec-1B by 4.04% in maximum swt and 5.43% in maximum ltr. This suggests limited DPO training can effectively aligns the model with desired generation patterns.

Third, the proposed IPA strategy outperforms various existing DPO variants. As shown in Table 1, IPA achieves superior performance compared to alternative DPO implementations. Notably, some DPO baselines underperform even the non-aligned OneRec-1B model, suggesting that iterative mining of self-generated outputs for preference selection proves more effective than other methods.

5.3 Ablation Study

5.3.1 DPO Sample Ratio Ablation. In order to investigate the impact of sample ratio r_{DPO} in DPO training, we varied the DPO sample ratio from 1% to 5% under controlled conditions. As illustrated in Figure 4, ablation results demonstrate that increasing the sample ratio yields marginal performance improvements across multiple evaluation targets. Notably, the performance gains beyond the 1% baseline remain insignificant despite increased computational expenditure. It worth noting that there exists a linear relationship between and GPU resource utilization during DPO sample server inference: the 5% sample ratio requires 5× more GPU resources than the 1% baseline. This scaling characteristic establishes an explicit trade-off between computational efficiency and model performance. Therefore, after balancing the best trade-off with computation efficiency and performance, we apply 1% DPO sample ratio for training, which achieves average 95% of the maximum observed performance while requiring only 20% of the computational resources needed for higher sample ratio.

5.3.2 Model Scaling Ablation. We evaluate how OneRec performs when the model scale increases. As Figure 6 shows, scaling OneRec from 0.05B to 1B achieves consistent accuracy gains, demonstrating consistent scaling properties. Specifically, compared to OneRec-0.05B, OneRec-0.1B achieves a significant maximum 14.45% gain in accuracy, and 5.09%, 5.70% and 5.69% additional accuracy gains can be achieved when scaling to 0.2B, 0,5B and 1B.

5.4 Prediction Dynamics of OneRec

As shown in Figure 5, we present the predicted probability distributions of 8192 codes across different layer, where the red star denotes the semantic ID of the item with the highest reward value.

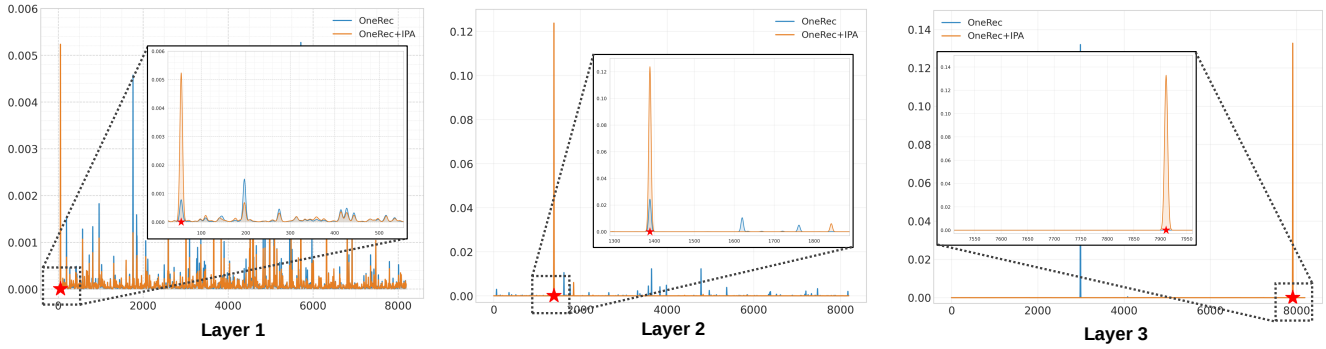

Figure 5: The visualization of the probability distribution of the softmax output for each layer of the semantic ID. The red star represents the sematic ID of item which has the highest reward value.

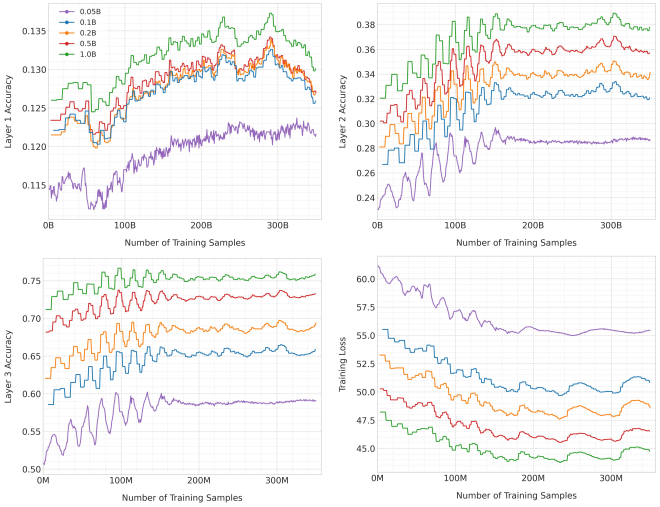

Figure 6: Scalability of OneRec on model scaling. The results show that OneRec constantly benefits from performance improvement when the parameters are scaled up.

Compared to the OneRec baseline, OneRec+IPA exhibits a significant confidence shift in prediction distributions, indicating that our proposed preference alignment strategy effectively encourages the base model to produce preferred generation patterns. Furthermore, we observe that the probability distribution in the first layer demonstrates greater divergence (entropy = 6.00) compared to subsequent layers (average entropy = 3.71 in the second layer and entropy = 0.048 in third layer), which exhibit progressively concentrated distributions. This hierarchical uncertainty reduction can be attributed to the autoregressive decoding mechanism: the initial layer's predictions inherit higher uncertainty from preceding decoding steps, while later layers benefit from accumulated context that constrains the decision space.

5.5 Online A/B Test

To evaluate the online performance of OneRec, we conduct strict online A/B tests on Kuaishou's video recommendation scenarios of main page and we compare the performance of OneRec and current multi-stage recommender system with 1% main traffic for experiments. We use *Total Watch Time* to measure the total time that users spend watching videos and *Average View Duration* calculates the average watch time per video when the user is exposed to a requested session by the recommendation system. Online evaluation shows that OneRec has achieved **1.68%** improvement in total watch time and **6.56%** improvement in average view duration, which indicates that OneRec achieves much better recommendation results and brings considerable revenue increments for the platform.

Table 2: The absolute improvement of OneRec compared to the current multi-stage system in the online A/B testing setting.

Model	Total Watch Time	Average View Duration
OneRec-0.1B	+0.57%	+4.26%
OneRec-1B	+1.21%	+5.01%
OneRec-1B+IPA	+1.68%	+6.56%

6 Conclusion

In this paper, we focus on the introduction of an industrial solution for single-stage generative recommendation. Our solution establishes three key contributions: First, we effectively scale the model parameters with high computational efficiency by applying the MoE architecture, offering a scalable blueprint for large-scale industrial recommendation. Next, we find the necessity of modeling the contextual information of target items in a session-wise generation manner, proving contextual sequence modeling inherently captures user preference dynamics better than isolated point-wise manner. Furthermore, we propose an Iterative Preference Alignment (IPA) strategy to improve OneRec's generalization across diverse user preference patterns. Extensive offline experiments and online A/B testing verify the effectiveness and efficiency of OneRec. Additionally, our analysis of online results reveals that, besides user watch time, our model has limitations in interactive indicators, such as likes. In future research, we aim to enhance the end-to-end generative recommendation's capability in multi-objective modeling to provide a better user experience.

References

[1] Mohammad Gheshlaghi Azar, Zhaohan Daniel Guo, Bilal Piot, Remi Munos, Mark Rowland, Michal Valko, and Daniele Calandriello. 2024. A general theoretical paradigm to understand learning from human preferences. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 4447–4455.

[2] Christopher JC Burges. 2010. From ranknet to lambdarank to lambdamart: An overview. *Learning* 11, 23-581 (2010), 81.

[3] Jianxin Chang, Chenbin Zhang, Zhiyi Fu, Xiaoxue Zang, Lin Guan, Jing Lu, Yiqun Hui, Dewei Leng, Yanan Niu, Yang Song, et al. 2023. TWIN: TWo-stage interest network for lifelong user behavior modeling in CTR prediction at kuaishou. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 3785–3794.

[4] Yuxin Chen, Junfei Tan, An Zhang, Zhengyi Yang, Leheng Sheng, Enzhi Zhang, Xiang Wang, and Tat-Seng Chua. 2024. On Softmax Direct Preference Optimization for Recommendation. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*. https://openreview.net/forum?id=qp5VbGTaM0

[5] Sayak Ray Chowdhury, Anush Kini, and Nagarajan Natarajan. 2024. Provably Robust DPO: Aligning Language Models with Noisy Feedback. In *ICML 2024*.

[6] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM conference on recommender systems*. 191–198.

[7] Damai Dai, Chengqi Deng, Chenggang Zhao, RX Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y Wu, et al. 2024. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066* (2024).

[8] Nicola De Cao, Gautier Izacard, Sebastian Riedel, and Fabio Petroni. 2020. Autoregressive entity retrieval. *arXiv preprint arXiv:2010.00904* (2020).

[9] Nan Du, Yanping Huang, Andrew M Dai, Simon Tong, Dmitry Lepikhin, Yuanzhong Xu, Maxim Krikun, Yanqi Zhou, Adams Wei Yu, Orhan Firat, et al. 2022. Glam: Efficient scaling of language models with mixture-of-experts. In *International Conference on Machine Learning*. PMLR, 5547–5569.

[10] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).

[11] Hongliang Fei, Jingyuan Zhang, Xingxuan Zhou, Junhao Zhao, Xinyang Qi, and Ping Li. 2021. GemNN: gating-enhanced multi-task neural networks with feature interaction learning for CTR prediction. In *Proceedings of the 44th international ACM SIGIR conference on research and development in information retrieval*. 2166–2171.

[12] Chao Feng, Wuchao Li, Defu Lian, Zheng Liu, and Enhong Chen. 2022. Recommender forest for efficient retrieval. *Advances in Neural Information Processing Systems* 35 (2022), 38912–38924.

[13] Luke Gallagher, Ruey-Cheng Chen, Roi Blanco, and J Shane Culpepper. 2019. Joint optimization of cascade ranking models. In *Proceedings of the twelfth ACM international conference on web search and data mining*. 15–23.

[14] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. 2013. Optimized product quantization. *IEEE transactions on pattern analysis and machine intelligence* 36, 4 (2013), 744–755.

[15] Huifeng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, and Xiuqiang He. 2017. DeepFM: a factorization-machine based neural network for CTR prediction. *arXiv preprint arXiv:1703.04247* (2017).

[16] B Hidasi. 2015. Session-based Recommendations with Recurrent Neural Networks. *arXiv preprint arXiv:1511.06939* (2015).

[17] Michael E Houle and Michael Nett. 2014. Rank-based similarity search: Reducing the dimensional dependence. *IEEE transactions on pattern analysis and machine intelligence* 37, 1 (2014), 136–150.

[18] Jiri Hron, Karl Krauth, Michael Jordan, and Niki Kilbertus. 2021. On component interactions in two-stage recommender systems. *Advances in neural information processing systems* 34 (2021), 2744–2757.

[19] Po-Sen Huang, Xiaodong He, Jianfeng Gao, Li Deng, Alex Acero, and Larry Heck. 2013. Learning deep structured semantic models for web search using clickthrough data. In *Proceedings of the 22nd ACM international conference on Information & Knowledge Management*. 2333–2338.

[20] Xu Huang, Defu Lian, Jin Chen, Liu Zheng, Xing Xie, and Enhong Chen. 2023. Cooperative Retriever and Ranker in Deep Recommenders. In *Proceedings of the ACM Web Conference 2023*. 1150–1161.

[21] Herve Jegou, Matthijs Douze, and Cordelia Schmid. 2010. Product quantization for nearest neighbor search. *IEEE transactions on pattern analysis and machine intelligence* 33, 1 (2010), 117–128.

[22] Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)*. IEEE, 197–206.

[23] Zhirui Kuai, Zuxu Chen, Huimu Wang, Mingming Li, Dadong Miao, Wang Binbin, Xusong Chen, Li Kuang, Yuxing Han, Jiaxing Wang, et al. 2024. Breaking the Hourglass Phenomenon of Residual Quantization: Enhancing the Upper Bound of Generative Retrieval. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*. 677–685.

[24] Doyup Lee, Chiheon Kim, Saehoon Kim, Minsu Cho, and Wook-Shin Han. 2022. Autoregressive image generation using residual quantization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 11523–11532.

[25] Han Liu, Yinwei Wei, Xuemeng Song, Weili Guan, Yuan-Fang Li, and Liqiang Nie. 2024. MMGRec: Multimodal Generative Recommendation with Transformer Model. *arXiv preprint arXiv:2404.16555* (2024).

[26] Shichen Liu, Fei Xiao, Wenwu Ou, and Luo Si. 2017. Cascade ranking for operational e-commerce search. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 1557–1565.

[27] Xinchen Luo, Jiangxia Cao, Tianyu Sun, Jinkai Yu, Rui Huang, Wei Yuan, Hezheng Lin, Yichen Zheng, Shiyao Wang, Qigen Hu, et al. 2024. QARM: Quantitative Alignment Multi-Modal Recommendation at Kuaishou. *arXiv preprint arXiv:2411.11739* (2024).

[28] Xu Ma, Pengjie Wang, Hui Zhao, Shaoguo Liu, Chuhan Zhao, Wei Lin, Kuang-Chih Lee, Jian Xu, and Bo Zheng. 2021. Towards a better tradeoff between effectiveness and efficiency in pre-ranking: A learnable feature selection based approach. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2036–2040.

[29] Yu Meng, Mengzhou Xia, and Danqi Chen. 2024. SimPO: Simple Preference Optimization with a Reference-Free Reward. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[30] Eric Mitchell. [n. d.]. A note on dpo with noisy preferences and relationship to ipo, 2023. *URL https://ericmitchell. ai/cdpo. pdf* ([n. d.]).

[31] Marius Muja and David G Lowe. 2014. Scalable nearest neighbor algorithms for high dimensional data. *IEEE transactions on pattern analysis and machine intelligence* 36, 11 (2014), 2227–2240.

[32] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in neural information processing systems* 35 (2022), 27730–27744.

[33] Qi Pi, Guorui Zhou, Yujing Zhang, Zhe Wang, Lejian Ren, Ying Fan, Xiaoqiang Zhu, and Kun Gai. 2020. Search-based user interest modeling with lifelong sequential behavior data for click-through rate prediction. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*. 2685–2692.

[34] Jiarui Qin, Jiachen Zhu, Bo Chen, Zhirong Liu, Weiwen Liu, Ruiming Tang, Rui Zhang, Yong Yu, and Weinan Zhang. 2022. Rankflow: Joint optimization of multi-stage cascade ranking systems as flows. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 814–824.

[35] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. 2024. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems* 36 (2024).

[36] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan Hulikal Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Tran, Jonah Samost, et al. 2023. Recommender systems with generative retrieval. *Advances in Neural Information Processing Systems* 36 (2023), 10299–10315.

[37] Wentao Shi, Jiawei Chen, Fuli Feng, Jizhi Zhang, Junkang Wu, Chongming Gao, and Xiangnan He. 2023. On the theories behind hard negative sampling for recommendation. In *Proceedings of the ACM Web Conference 2023*. 812–822.

[38] Anshumali Shrivastava and Ping Li. 2014. Asymmetric LSH (ALSH) for sublinear time maximum inner product search (MIPS). *Advances in neural information processing systems* 27 (2014).

[39] Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F Christiano. 2020. Learning to summarize with human feedback. *Advances in Neural Information Processing Systems* 33 (2020), 3008–3021.

[40] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the 28th ACM international conference on information and knowledge management*. 1441–1450.

[41] Yubao Tang, Ruqing Zhang, Jiafeng Guo, and Maarten de Rijke. 2023. Recent advances in generative information retrieval. In *Proceedings of the Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region*. 294–297.

[42] Yi Tay, Vinh Tran, Mostafa Dehghani, Jianmo Ni, Dara Bahri, Harsh Mehta, Zhen Qin, Kai Hui, Zhe Zhao, Jai Gupta, et al. 2022. Transformer memory as a differentiable search index. *Advances in Neural Information Processing Systems* 35 (2022), 21831–21843.

[43] Lidan Wang, Jimmy Lin, and Donald Metzler. 2011. A cascade ranking model for efficient ranked retrieval. In *Proceedings of the 34th international ACM SIGIR conference on Research and development in Information Retrieval*. 105–114.

[44] Yunli Wang, Zhiqiang Wang, Jian Yang, Shiyang Wen, Dongying Kong, Han Li, and Kun Gai. 2024. Adaptive Neural Ranking Framework: Toward Maximized Business Goal for Cascade Ranking Systems. In *Proceedings of the ACM on Web Conference 2024*. 3798–3809.

[45] Ye Wang, Jiahao Xun, Minjie Hong, Jieming Zhu, Tao Jin, Wang Lin, Haoyuan Li, Linjun Li, Yan Xia, Zhou Zhao, et al. 2024. EAGER: Two-Stream Generative Recommender with Behavior-Semantic Collaboration. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 3245–3254.

[46] Zhe Wang, Liqin Zhao, Biye Jiang, Guorui Zhou, Xiaoqiang Zhu, and Kun Gai. 2020. Cold: Towards the next generation of pre-ranking system. *arXiv preprint arXiv:2007.16122* (2020).

[47] Haoran Xu, Amr Sharaf, Yunmo Chen, Weiting Tan, Lingfeng Shen, Benjamin Van Durme, Kenton Murray, and Young Jin Kim. 2024. Contrastive preference optimization: Pushing the boundaries of llm performance in machine translation. *arXiv preprint arXiv:2401.08417* (2024).

[48] Shuyuan Xu, Wenyue Hua, and Yongfeng Zhang. 2024. Openp5: An open-source platform for developing, training, and evaluating llm-based recommender systems. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 386–394.

[49] Neil Zeghidour, Alejandro Luebs, Ahmed Omran, Jan Skoglund, and Marco Tagliasacchi. 2021. Soundstream: An end-to-end neural audio codec. *IEEE/ACM Transactions on Audio, Speech, and Language Processing* 30 (2021), 495–507.

[50] Tingting Zhang, Pengpeng Zhao, Yanchi Liu, Victor S Sheng, Jiajie Xu, Deqing Wang, Guanfeng Liu, Xiaofang Zhou, et al. 2019. Feature-level deeper self-attention network for sequential recommendation.. In *IJCAI*. 4320–4326.

[51] Bowen Zheng, Yupeng Hou, Hongyu Lu, Yu Chen, Wayne Xin Zhao, Ming Chen, and Ji-Rong Wen. 2024. Adapting large language models by integrating collaborative semantics for recommendation. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 1435–1448.

[52] Guorui Zhou, Na Mou, Ying Fan, Qi Pi, Weijie Bian, Chang Zhou, Xiaoqiang Zhu, and Kun Gai. 2019. Deep interest evolution network for click-through rate prediction. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 33. 5941–5948.

[53] Guorui Zhou, Xiaoqiang Zhu, Chenru Song, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, and Kun Gai. 2018. Deep interest network for click-through rate prediction. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 1059–1068.

[54] Han Zhu, Xiang Li, Pengye Zhang, Guozheng Li, Jie He, Han Li, and Kun Gai. 2018. Learning tree-based deep model for recommender systems. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 1079–1088.

[55] Barret Zoph, Irwan Bello, Sameer Kumar, Nan Du, Yanping Huang, Jeff Dean, Noam Shazeer, and William Fedus. 2022. Designing effective sparse expert models. *arXiv preprint arXiv:2202.08906* 2, 3 (2022), 17.